# Lane2 Language Specification

Version: v1 draft
Status: working specification

## Contents


1 Introduction ..... 2
1.1 Scope ..... 2
1.2 Compatibility ..... 2
1.3 Experimental Features ..... 3
1.4 Feedback ..... 3
1.5 Reference ..... 3
2 Syntax and Grammar ..... 3
2.1 Notation ..... 3
2.2 Lexical Grammar ..... 3
2.3 Syntax Grammar ..... 6
2.4 Comments ..... 10
3 Type System ..... 12
3.1 Notations ..... 12
3.2 Kinds ..... 14
3.3 Primitive Types ..... 14
3.4 Function Types ..... 15
3.5 Generic Function Types ..... 16
3.6 Nominal Type Constructors ..... 17
3.7 Existential Packages ..... 18
3.8 Struct Types ..... 18
3.9 Enum Types ..... 20
4 Type Inference ..... 21
5 Builtins ..... 21
5.1 Required Intrinsics ..... 22
6 Prelude ..... 22
6.1 Standard Prelude Source ..... 22
7 Declarations ..... 26
7.1 Top-Level Declarations ..... 26
7.2 Struct Declarations ..... 26
7.3 Enum Declarations ..... 26
7.4 Function Declarations ..... 26
7.5 Let Declarations ..... 27
8 Scopes and Bindings ..... 28
8.1 Notations ..... 28
8.2 Resolution Model ..... 28
8.3 Namespaces ..... 29
8.4 Top-Level Scope ..... 29
8.5 Local Scope ..... 30
8.6 Contextual Offers ..... 31
8.7 Contextual Forwarding Fields ..... 32
8.8 Direct Named Calls and Contextual Arguments ..... 32
8.9 Special Resolution Forms ..... 33
9 Expressions ..... 33
9.1 Notations ..... 34
9.2 Blocks ..... 34
9.3 Conditional Expressions ..... 35
9.4 Calls, Field Access, and Pipeline ..... 35

9.5 Struct and Enum Expressions ..... 37
9.6 Match Expressions and Patterns ..... 38
9.7 Operator Expressions ..... 40
10 Buslane Core Language ..... 41
10.1 Syntax ..... 41
10.2 Typing ..... 44
10.3 Evaluation ..... 47
10.4 Relation to ANF ..... 49
11 Evaluation ..... 49
12 Undefined Behavior ..... 49
13 Conformance ..... 49
14 Appendix ..... 49
14.1 Deliberately Omitted From V1 ..... 49

# 1 Introduction

Lane2 is a strict, pure, expression-oriented functional programming language. Its first implementation target is a parser, a type checker, and an AST interpreter; later implementations may add bytecode compilation, a virtual machine, a linker, and effect handling.

Lane2 is intentionally small. It has no mutable state, assignment, trait system, method dispatch, module system, or implicit typeclass-style instance search in v1. Instead, v1 is centered on nominal data, first-class functions, bidirectional local type inference, exhaustive pattern matching, and contextual resolution of explicitly offered values.

This document specifies Lane2/Core v1: the platform-independent part of Lane2 that should behave the same across the initial AST interpreter and later execution backends.

## 1.1 Scope

Lane2/Core v1 covers:

- source structure and declarations;
- lexical scope and name resolution;
- nominal struct and enum types, including existential type members and variant-local hidden type binders;
- function and block expressions;
- local type inference;
- pattern matching;
- contextual resolution and operation-based operators;
- unsafe builtin expressions.

The following are outside Lane2/Core v1:

- mutation and assignment;
- modules and imports;
- bytecode and virtual machine semantics;
- linker entrypoint selection;
- algebraic effects;
- type aliases;
- traits, typeclasses, and interfaces;
- tuples and collection syntax.

## 1.2 Compatibility

This specification is a v1 draft. No compatibility is promised between draft revisions. Language rules may be added, removed, or changed while the first implementation is being developed.

Once a v1 implementation is declared stable, this section should be replaced by an explicit compatibility policy.

### 1.3 Experimental Features

This draft does not distinguish stable and experimental language features. Every rule in this document is provisional until v1 is stabilized.

Future drafts may mark individual features as experimental when their surface syntax or semantics are intentionally unsettled.

### 1.4 Feedback

Design feedback is tracked in the Lane2 repository. Implementation changes should update this specification, `CONTEXT.md`, and ADRs when the change affects user-visible semantics.

### 1.5 Reference

When referencing this draft, use:

> Lane2 Language Specification : Lane2/Core v1 draft.

## 2 Syntax and Grammar

### 2.1 Notation

```
grammar        ::= { production }
production     ::= nonTerminal "::=" grammarExpression
grammarExpression ::=
    grammarSequence { "|" grammarSequence }
grammarSequence ::=
    { grammarItem }
grammarItem ::=
    terminal
  | nonTerminal
  | "[" grammarExpression "]"
  | "{" grammarExpression "}"
  | "(" grammarExpression ")"
  | grammarItem "?"
  | grammarItem "*"
  | grammarItem "+"

terminal       ::= "'" terminalText "'"
nonTerminal    ::= lowerCamelIdentifier

empty sequence denotes epsilon.
"A B" denotes sequencing.
"A | B" denotes choice.
"[ A ]" and "A?" denote optional occurrence.
"{ A }" and "A*" denote zero or more occurrences.
"A+" denotes one or more occurrences.
Parenthesized grammar expressions group without producing syntax.
```

### 2.2 Lexical Grammar

```
lexicalInput ::=
    (trivia | token)* eof

trivia ::=
    whitespace
  | lineTerminator
  | comment

token ::=
    keyword
  | reservedWord
```

```
  | identifier
  | intLiteral
  | stringLiteral
  | boolLiteral
  | operatorToken
  | punctuationToken

keyword ::=
    "struct"
  | "enum"
  | "fn"
  | "let"
  | "auto"
  | "offer"
  | "if"
  | "else"
  | "match"
  | "builtin"

reservedWord ::=
    "effect"
  | "handler"
  | "module"
  | "import"
  | "pub"
  | "type"
  | "return"

identifier ::=
    asciiIdentifierStart identifierContinue*
  | "_" identifierContinue+

identifierContinue ::=
    "_"
  | asciiIdentifierStart
  | decimalDigit

boolLiteral ::=
    "true"
  | "false"

intLiteral ::=
    decimalDigit+

stringLiteral ::=
    "\"" stringElement* "\""

stringElement ::=
    asciiStringCharacter
  | stringEscape

asciiStringCharacter ::=
    asciiCodePointExceptControlBackslashQuote

stringEscape ::=
    "\\" "\\"
  | "\\" "\""
  | "\\" "n"
  | "\\" "r"
  | "\\" "t"
  | "\\" "x" asciiHexByte
```

```
asciiHexByte ::=
    asciiHexLowByte
  | asciiHexHighByte

asciiHexLowByte ::=
    ("0" | "1" | "2" | "3" | "4" | "5" | "6") asciiHexDigit

asciiHexHighByte ::=
    "7" ("0" | "1" | "2" | "3" | "4" | "5" | "6" | "7")

operatorToken ::=
    "+"
  | "-"
  | "*"
  | "/"
  | "%"
  | "=="
  | "!="
  | "<"
  | "<="
  | ">"
  | ">="
  | "&&"
  | "||"
  | "!"
  | "|>"

punctuationToken ::=
    "("
  | ")"
  | "{"
  | "}"
  | "["
  | "]"
  | ","
  | "."
  | ":"
  | "::"
  | "->"
  | "=>"
  | "="
  | "_"

whitespace ::=
    U+0009
  | U+000B
  | U+000C
  | U+0020

lineTerminator ::=
    U+000A
  | U+000D
  | U+000D U+000A

decimalDigit ::=
    "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

asciiHexDigit ::=
    decimalDigit
  | "a" | "b" | "c" | "d" | "e" | "f"
```

```
  | "A" | "B" | "C" | "D" | "E" | "F"

asciiIdentifierStart ::=
    asciiUppercaseLetter
  | asciiLowercaseLetter

asciiUppercaseLetter ::=
    any ASCII code point from U+0041 through U+005A

asciiLowercaseLetter ::=
    any ASCII code point from U+0061 through U+007A
```

Lexical analysis uses maximal munch. If more than one token class matches the same maximal source span, keyword, reservedWord, and boolLiteral take priority over identifier.

### 2.3 Syntax Grammar

```
sourceFile ::=
    topLevelDeclaration* eof

topLevelDeclaration ::=
    structDeclaration
  | enumDeclaration
  | functionDeclaration
  | topLevelLetDeclaration
  | offerValueDeclaration

structDeclaration ::=
    "struct" typeName typeParameters? "{" structTypeMember* structFieldDeclaration* "}"

structTypeMember ::=
    "type" typeName space ":" space kind

structFieldDeclaration ::=
    fieldModifier? fieldName space ":" space type

fieldModifier ::=
    "offer"

enumDeclaration ::=
    "enum" typeName typeParameters? "{" enumVariant* "}"

enumVariant ::=
    variantName typeParameters? enumPayload?

enumPayload ::=
    "(" commaSeparatedTypes? ")"

functionDeclaration ::=
    "fn" typeParameters? functionName "(" commaSeparatedParameters? ")" "->" type block

parameter ::=
    parameterModifiers? valueName space ":" space type

parameterModifiers ::=
    parameterModifier+

parameterModifier ::=
    "auto"
  | "offer"
```

```
topLevelLetDeclaration ::=
    "let" valueName space ":" space type "=" expression

localLetDeclaration ::=
    "let" valueName typeAnnotation? "=" expression

localPatternLetDeclaration ::=
    "let" pattern "=" expression

typeAnnotation ::=
    space ":" space type

offerValueDeclaration ::=
    "offer" valueName space ":" space type "=" expression

type ::=
    typeConstructor typeArguments?
  | functionType

typeConstructor ::=
    typeName

typeArguments ::=
    "[" commaSeparatedTypes "]"

typeParameters ::=
    "[" commaSeparatedTypeParameters "]"

kind ::=
    "Type"
  | "Type" "->" kind
  | "(" kind ")"

functionType ::=
    typeParameters? "(" commaSeparatedTypes? ")" "->" type

expression ::=
    ifExpression
  | matchExpression
  | functionLiteral
  | block
  | pipelineExpression

pipelineExpression ::=
    binaryExpression { "|>" pipelineRhs }

pipelineRhs ::=
    callExpression
  | functionLiteral

binaryExpression ::=
    unaryExpression { binaryOperator unaryExpression }

unaryExpression ::=
    unaryOperator unaryExpression
  | callExpression

callExpression ::=
    fieldExpression { callSuffix }

callSuffix ::=
```

```
    "(" commaSeparatedCallArguments? ")"

callArgument ::=
    expression
  | valueName "=" expression

fieldExpression ::=
    primaryExpression { "." fieldName }

primaryExpression ::=
    literal
  | valueName
  | qualifiedVariantExpression
  | structLiteral
  | builtinExpression
  | "(" expression ")"
  | "(" ")"

literal ::=
    intLiteral
  | stringLiteral
  | boolLiteral

qualifiedVariantExpression ::=
    typeName typeArguments? "::" variantName variantTypeArguments? variantArguments?

variantTypeArguments ::=
    "[" commaSeparatedTypes "]"

variantArguments ::=
    "(" commaSeparatedExpressions? ")"

structLiteral ::=
    typeName typeArguments? "::" "{" commaSeparatedStructLiteralFields "}"

structLiteralField ::=
    fieldName
  | fieldName ":" expression
  | typeName "=" type

builtinExpression ::=
    "builtin" "(" stringLiteral ")"

functionLiteral ::=
    "fn" typeParameters? "(" commaSeparatedFunctionLiteralParameters? ")"
functionReturnAnnotation? block

functionLiteralParameter ::=
    functionLiteralParameterModifiers? valueName
  | functionLiteralParameterModifiers? valueName space ":" space type

functionLiteralParameterModifiers ::=
    "offer"+

functionReturnAnnotation ::=
    "->" type

block ::=
    "{" localItem* expression "}"

localItem ::=
```

```
    localLetDeclaration
  | localPatternLetDeclaration
  | functionDeclaration
  | offerValueDeclaration

ifExpression ::=
    "if" expression block "else" block

matchExpression ::=
    "match" expression "{" matchArm* "}"

matchArm ::=
    pattern "=>" expression

pattern ::=
    "_"
  | valueName
  | literal
  | qualifiedVariantPattern
  | structPattern

qualifiedVariantPattern ::=
    typeName "::" variantName variantTypeBinders? patternArguments?

variantTypeBinders ::=
    "[" commaSeparatedTypeBinders "]"

patternArguments ::=
    "(" commaSeparatedPatterns? ")"

structPattern ::=
    typeName "::" "{" commaSeparatedStructPatternFields "}"

structPatternField ::=
    fieldName
  | typeName
  | "_"
  | fieldName ":" pattern

binaryOperator ::=
    "+" | "-" | "*" | "/" | "%"
  | "==" | "!=" | "<" | "<=" | ">" | ">="
  | "&&" | "||"

unaryOperator ::=
    "-" | "!"

commaSeparatedTypes ::=
    type ("," type)* ","?

commaSeparatedTypeParameters ::=
    typeParameter ("," typeParameter)* ","?

commaSeparatedTypeBinders ::=
    typeBinder ("," typeBinder)* ","?

commaSeparatedParameters ::=
    parameter ("," parameter)* ","?

commaSeparatedExpressions ::=
    expression ("," expression)* ","?
```

```
commaSeparatedCallArguments ::=
    callArgument ("," callArgument)* ","?

commaSeparatedFunctionLiteralParameters ::=
    functionLiteralParameter ("," functionLiteralParameter)* ","?

commaSeparatedStructLiteralFields ::=
    structLiteralField ("," structLiteralField)* ","?

commaSeparatedPatterns ::=
    pattern ("," pattern)* ","?

commaSeparatedStructPatternFields ::=
    structPatternField ("," structPatternField)* ","?

typeName ::=
    identifier

functionName ::=
    identifier

valueName ::=
    identifier

fieldName ::=
    identifier

variantName ::=
    identifier

typeParameter ::=
    identifier

typeBinder ::=
    identifier

space ::=
    whitespace+
```

The precedence and associativity of `binaryOperator`, `unaryOperator`, calls, field access, and pipeline expressions are defined in “Operator Expressions”.

## 2.4 Comments

```
comment ::=
    lineComment
  | blockComment

lineComment ::=
    "/" "/" lineCommentCharacter* lineTerminator?

lineCommentCharacter ::=
    any Unicode code point except U+000A or U+000D

blockComment ::=
    "/" "*" blockCommentCharacter* "*" "/"

blockCommentCharacter ::=
    any Unicode code point sequence that does not start "*/"
```

Comments are trivia. Comments do not appear in the syntactic grammar.

# 3 Type System

## 3.1 Notations

| Notation | Meaning |
|---|---|
| $T, U, R, F, P, Q$ | types |
| $A$ | universal type variable |
| $B$ | existential type variable |
| $K$ | kind |
| $C$ | type constructor |
| $S$ | struct type constructor |
| $E$ | enum type constructor |
| $e, c, f, b$ | expressions |
| $a$ | match arm |
| $x$ | value binder |
| $i, j, k, n, m, q$ | indices and natural numbers |
| $l$ | struct field name |
| $v$ | enum variant name |
| $\Delta$ | type-constructor context |
| $\Theta$ | type-variable context |
| $\Gamma$ | value typing context |
| $D$ | custom type definition |
| $C \to D \in \Delta$ | visible custom type definition binding |
| $\mathcal{S}(A_1, ..., A_n; B_1 : K_1, ..., B_r : K_r; l_1 : F_1, ..., l_m : F_m)$ | struct definition with universal parameters, existential type members, and value fields |
| $\mathcal{E}\left(A_1, ..., A_n; v_j\left[B_{j,1}:K_{j,1},...,B_{j,r_j}:K_{j,r_j}\right]\left(P_{j,1}, ..., P_{j,m_j}\right)\right)$ | enum definition with universal parameters and variant-local existential binders |
| $\pi(D) = (A_1, ..., A_n)$ | custom type parameters |
| $D \vdash v : (B_1 : K_1, ..., B_r : K_r; P_1, ..., P_m)$ | variant hidden binders and payload sequence declared by a custom type definition |
| $D \vdash \nu(v_1, ..., v_q)$ | variant sequence declared by a custom type definition |
| $\Delta\Theta \vdash E[T_1, ..., T_n] :: v[U_1, ..., U_r] \to (Q_1, ..., Q_m)$ | instantiated variant payload sequence |
| $\sigma$ | type substitution environment |
| $\sigma = [T_1/A_1, ..., T_n/A_n]$ | substitution environment binding |
| $\rho = [U_1/B_1, ..., U_r/B_r]$ | existential witness substitution |
| $U[T_1/A_1, ..., T_n/A_n]$ | direct type substitution |
| $U\sigma$ | type substitution by environment |
| $\Delta\Theta \vdash T : \text{Type}$ | well-formed type |
| $C[T_1, ..., T_n]$ | nominal type application |
| $\forall A_1, ..., A_n.T$ | universal type |
| $\exists B_1 : K_1, ..., B_r : K_r.T$ | existential package model |
| $T \equiv U$ | type equality |
| $\Gamma \vdash e : T$ | expression typing |
| $H(T)$ | type that mentions no unopened existential binder |
| $N(B_1, ..., B_r; T)$ | none of the listed existential binders occurs free in type $T$ |
| $\sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1, ..., B_r : K_r, P_1, ..., P_m)$ | instantiated value payload sequence for the fields of $S[T_1, ..., T_n]$ |

Table 1: Type system notations and meanings

## 3.2 Kinds

Lane2 v1 uses `Type` as the kind of ordinary value-level types. Type members carry explicit kinds so that the same surface form can later extend to higher-kinded witnesses.

**Kind formation.**

```
Type
Type -> Type
```

$$\Delta\Theta \vdash \text{Type} \checkmark$$

$$\frac{\Delta\Theta \vdash K_1 \checkmark \quad \Delta\Theta \vdash K_2 \checkmark}{\Delta\Theta \vdash K_1 \to K_2 \checkmark}$$

Declarations that use type members must record the written kind. `Type` is the first required kind, and function kinds such as `Type -> Type` are specified so existential type members are higher-kind-ready.

## 3.3 Primitive Types

Lane2/Core v1 has four primitive types: `Unit`, `Bool`, `Int`, and `String`.

**Primitive formation.** Each primitive type is well-formed in every type environment.

$$\Delta\Theta \vdash \text{Unit} : \text{Type}$$

$$\Delta\Theta \vdash \text{Bool} : \text{Type}$$

$$\Delta\Theta \vdash \text{Int} : \text{Type}$$

$$\Delta\Theta \vdash \text{String} : \text{Type}$$

**Unit introduction.** The expression `()` introduces a `Unit` value.

```
() // unit literal
```

$$\Gamma \vdash () : \text{Unit}$$

**Unit elimination.** Lane2/Core v1 has no dedicated elimination form for `Unit`.

**Bool introduction.** Boolean literals introduce `Bool` values.

```
true  // boolean literal
false // boolean literal
```

$$\Gamma \vdash \text{true} : \text{Bool}$$

$$\Gamma \vdash \text{false} : \text{Bool}$$

**Bool elimination.** `if` eliminates a `Bool` value.

```
if c {
  e1
```

```
} else {
  e2
} // c : Bool, e1 : T, e2 : T
```

$$\frac{\Gamma \vdash c : \text{Bool} \quad \Gamma \vdash e_1 : T \quad \Gamma \vdash e_2 : T}{\Gamma \vdash \text{if } c \text{ then } e_1 \text{ else } e_2 : T}$$

**Int introduction.** Integer literals introduce `Int` values.

```
n // integer literal
```

$$\Gamma \vdash n : \text{Int}$$

**Int elimination.** Lane2/Core v1 has no built-in syntactic eliminator for `Int`. Integer operations are typed through required intrinsics and prelude operation values.

**String introduction.** ASCII string literals introduce `String` values.

```
"..." // ASCII string literal
```

$$\Gamma \vdash s : \text{String}$$

**String elimination.** Lane2/Core v1 has no built-in syntactic eliminator for `String`. String equality is typed through the prelude.

**Primitive type equality.** Primitive types are equal only to themselves.

$$\text{Unit} \equiv \text{Unit}$$

$$\text{Bool} \equiv \text{Bool}$$

$$\text{Int} \equiv \text{Int}$$

$$\text{String} \equiv \text{String}$$

## 3.4 Function Types

**Function formation.**

```
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
let f : (T1, ..., Tn) -> R = fn(x1, ..., xn) { e } // function literal binding
```

$$\frac{\Delta\Theta \vdash T_1 : \text{Type} \quad \ldots \quad \Delta\Theta \vdash T_n : \text{Type} \quad \Delta\Theta \vdash R : \text{Type}}{\Delta\Theta \vdash (T_1, \ldots, T_n) \to R : \text{Type}}$$

**Function introduction.**

```
fn(x1 : T1, ..., xn : Tn) -> R { e } // function literal
fn f(x1 : T1, ..., xn : Tn) -> R { e } // function definition
```

$$\frac{\Gamma, x_1 : T_1, ..., x_n : T_n \vdash e : R}{\Gamma \vdash \text{fn } (x_1 : T_1, ..., x_n : T_n) \to R \ \{ \ e \ \} : (T_1, ..., T_n) \to R}$$

**Function elimination.**

```
f(a1, ..., an) // function call
```

$$\frac{\Gamma \vdash f : (T_1, ..., T_n) \to R \quad \Gamma \vdash a_1 : T_1 \quad ... \quad \Gamma \vdash a_n : T_n}{\Gamma \vdash f(a_1, ..., a_n) : R}$$

**Function type equality.**

$$\frac{T_1 \equiv U_1 \quad ... \quad T_n \equiv U_n \quad R \equiv S}{(T_1, ..., T_n) \to R \equiv (U_1, ..., U_n) \to S}$$

Function types are uncurried. `(T1, T2) -> R` is not the same type object as `(T1) -> (T2) -> R`.

## 3.5 Generic Function Types

**Generic function formation.**

```
[A1, ..., An](T1, ..., Tm) -> R // generic function type
```

$$\frac{\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R : \text{Type}}{\Delta\Theta \vdash \forall A_1, ..., A_n.(T_1, ..., T_m) \to R : \text{Type}}$$

**Generic function introduction.**

```
fn[A1, ..., An](x1 : T1, ..., xm : Tm) -> R { e } // generic function literal
fn[A1, ..., An] f(x1 : T1, ..., xm : Tm) -> R { e } // generic function definition
```

A generic function literal or named generic function introduces a generic function value. The function body is checked under the declared type parameters and value parameters.

$$\frac{\begin{cases}\Delta\Theta, A_1, ..., A_n \vdash (T_1, ..., T_m) \to R : \text{Type} \\ \Gamma, x_1 : T_1, ..., x_m : T_m \vdash e : R\end{cases}}{\Gamma \vdash \text{fn } [ \ A_1, ..., A_n \ ] \ (x_1 : T_1, ..., x_m : T_m) \to R \ \{ \ e \ \} : \forall A_1, ..., A_n.(T_1, ..., T_m) \to R}$$

**Generic function elimination.**

Generic function calls instantiate type parameters at the use site when the use site is unambiguous.

```
f(a1, ..., am) // generic function call with inferred type arguments
```

$$\frac{\begin{cases}\Gamma \vdash f : \forall A_1, ..., A_n.(T_1, ..., T_m) \to R \\ \sigma = [U_1/A_1, ..., U_n/A_n] \\ \forall i.\Gamma \vdash a_i : T_i\sigma\end{cases}}{\Gamma \vdash f(a_1, ..., a_m) : R\sigma}$$

The substitution $\sigma$ must be the unique substitution selected by local type inference for the call site. If no unique substitution exists, the call is ill-typed.

**Generic function type equality.**

$$\frac{(T_1, ..., T_m) \to R \equiv (U_1, ..., U_m) \to S}{\forall A_1, ..., A_n.(T_1, ..., T_m) \to R \equiv \forall A_1, ..., A_n.(U_1, ..., U_m) \to S}$$

Generic function type equality compares the number of type parameters and the function types under corresponding bound type parameters.

## 3.6 Nominal Type Constructors

Struct and enum declarations introduce nominal custom type definitions in Δ. This section specifies the shared treatment of nominal type constructors. Struct-specific and enum-specific declaration rules are specified in "Struct Types" and "Enum Types".

Top-level custom type declarations are checked in two phases. First, all top-level struct and enum constructors are collected into Δ as declaration-shape bindings. Second, every type member kind, field type, variant-local type binder kind, and variant payload type is checked under the collected Δ and the declaration's type parameters. This permits mutually recursive custom types.

**Custom type collection.** A top-level custom type declaration contributes its nominal constructor and declaration shape to Δ before member type checking begins. In struct declaration shapes, $B_1 : K_1, ..., B_r : K_r$ denotes hidden type members. In enum declaration shapes, $B_{j,1} : K_{j,1}, ..., B_{j,r_j} : K_{j,r_j}$ denotes variant-local hidden type binders, and $P_j$ denotes the payload type sequence of variant $v_j$.

```
struct S[A1, ..., An] {
  type B1 : K1
  ...
  l1 : F1
  ...
  lm : Fm
}

enum E[A1, ..., An] {
  vj[Bj1, ..., Bjrj](Pj1, ..., Pjmj)
}
```

$$S \to \mathcal{S}(A_1, ..., A_n; B_1 : K_1, ..., B_r : K_r; l_1 : F_1, ..., l_m : F_m) \in \Delta$$

$$E \to \mathcal{E}\left(A_1, ..., A_n; v_1\left[B_{1,1} : K_{1,1}, ..., B_{1,r_1} : K_{1,r_1}\right](P_1), ..., v_{q\left[B_{q,1}:K_{q,1},...,B_{q,r_q}:K_{q,r_q}\right]}(P_q)\right) \in \Delta$$

**Nominal formation.** A nominal type application is well-formed when its custom type constructor is visible, the number of type arguments matches the declaration's type parameters, and every type argument is well-formed.

$$\frac{\begin{cases} C \to D \in \Delta \\ \pi(D) = (A_1, ..., A_n) \\ \forall i. \Delta\Theta \vdash T_i : \text{Type} \end{cases}}{\Delta\Theta \vdash C[T_1, ..., T_n] : \text{Type}}$$

**Nominal type equality.** Nominal type equality compares constructor identity and then compares type arguments pairwise. There is no equality rule whose conclusion relates different nominal constructors.

$$\frac{T_1 \equiv U_1 \quad \ldots \quad T_n \equiv U_n}{C[T_1, \ldots, T_n] \equiv C[U_1, \ldots, U_n]}$$

Existential type members and variant-local binders are not arguments of the nominal type constructor. They do not participate in nominal type equality; their witnesses are hidden inside constructed values and can be opened only by pattern-based elimination.

## 3.7 Existential Packages

Lane2 surface structs and enum variants with hidden type binders are modeled by existential packages. The existential type form is a static model used by the specification; Lane2 source does not provide a general `exists` type syntax.

**Existential formation.**

$$\frac{\Delta\Theta, B_1 : K_1, \ldots, B_r : K_r \vdash T : \mathrm{Type}}{\Delta\Theta \vdash \exists B_1 : K_1, \ldots, B_r : K_r.T : \mathrm{Type}}$$

**Existential introduction.**

```
Hide::{ T = Int, val: 5 } // struct package
Hide::hide[Int](5)        // enum variant package
```

$$\frac{\begin{cases} \forall p.\Delta\Theta \vdash U_p : K_p \\ \rho = [U_1/B_1, \ldots, U_r/B_r] \\ \Gamma \vdash v : T\rho \end{cases}}{\Gamma \vdash \mathrm{pack}\ [\ U_1, \ldots, U_r\ ]\ v : \exists B_1 : K_1, \ldots, B_r : K_r.T}$$

**Existential elimination.**

```
let Hide::{ T, val } = h
match h {
  Hide::hide[T](val) => body
}
```

$$\frac{\Gamma \vdash e : \exists B : K.T \quad \Gamma, B : K, x : T \vdash b : R \quad N(B; R)}{\Gamma \vdash \mathrm{unpack}\ e\ \mathrm{as}\ [\ B, x\ ]\ \mathrm{in}\ b : R}$$

The side condition prevents an opened hidden type from escaping the scope that opened it. A value whose type mentions an opened hidden type must be repacked into another existential before leaving that scope.

## 3.8 Struct Types

**Struct declaration well-formedness.** A struct declaration is well-formed when every declared type member kind is well-formed and every declared field type is well-formed under the collected type-constructor context, the struct's universal type parameters, and the struct's hidden type members. Type members must appear before value fields in source.

```
struct S[A1, ..., An] {
  type B1 : K1
  ...
  l1 : F1
  ...
```

```
  lm : Fm
} // struct definition
```

$$\frac{\begin{cases} S \to \mathcal{S}(A_1, ..., A_n; B_1 : K_1, ..., B_r : K_r; l_1 : F_1, ..., l_m : F_m) \in \Delta \\ \forall p. \Delta A_1, ..., A_n \vdash K_p \checkmark \\ \forall i. \Delta A_1, ..., A_n, B_1 : K_1, ..., B_r : K_r \vdash F_i : \text{Type} \end{cases}}{\Delta \vdash \text{struct } S \ [\ A_1, ..., A_n\ ] \ \{\ B_1 : K_1, ..., B_r : K_r; l_1 : F_1, ..., l_m : F_m\ \} \checkmark}$$

**Struct introduction.** A qualified struct literal introduces a struct value. It must provide every declared type-member witness and every declared value field exactly once. Source order is not significant for named witnesses and fields; the rule below uses declaration order.

```
S[T1, ..., Tn]::{ l1: e1, ..., lm: em } // qualified struct literal
Hide::{ B1 = U1, val: e } // qualified struct literal with hidden type witness
```

$$\frac{S \to \mathcal{S}(A_1, ..., A_n; B_1 : K_1, ..., B_r : K_r; l_1 : F_1, ..., l_m : F_m) \in \Delta \quad \sigma = [T_1/A_1, ..., T_n/A_n]}{\sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1\sigma, ..., B_r : K_r\sigma l_1 : F_1\sigma, ..., l_m : F_m\sigma)}$$

$$\frac{\begin{cases} \sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1, ..., B_r : K_r; l_1 : Q_1, ..., l_m : Q_m) \\ \forall p. \Delta\Theta \vdash U_p : K_p \\ \rho = [U_1/B_1, ..., U_r/B_r] \\ \forall i. \Gamma \vdash e_i : Q_i\rho \end{cases}}{\Gamma \vdash S \ [\ T_1, ..., T_n\ ] ::\{\ B_1 = U_1, ..., B_r = U_r, l_1 : e_1, ..., l_m : e_m\ \} : S[T_1, ..., T_n]}$$

**Struct field elimination.** Field access eliminates a struct value by selecting a declared field type after substituting the struct type arguments. Field access alone does not open hidden type members. Therefore the selected field type must not mention unopened hidden type members.

```
e.l // field access
```

$$\frac{\Gamma \vdash e : S[T_1, ..., T_n] \quad \sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1, ..., B_r : K_r; ..., l : Q, ...) \quad H(Q)}{\Gamma \vdash e.l : Q}$$

**Struct pattern elimination.** A struct pattern may open hidden type members and bind value fields. The opened type binders are abstract and are available only in the scope governed by the pattern.

```
let S::{ B1, ..., Br, l1: x1, ..., lm: xm } = e
```

$$\frac{\begin{cases} \Gamma \vdash e : S[T_1, ..., T_n] \\ \sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1, ..., B_r : K_r; l_1 : Q_1, ..., l_m : Q_m) \\ \Gamma, B_1 : K_1, ..., B_r : K_r, x_1 : Q_1, ..., x_m : Q_m \vdash b : R \\ N(B_1, ..., B_r; R) \end{cases}}{\Gamma \vdash \text{let } S ::\{\ B_1, ..., B_r, l_1 : x_1, ..., l_m : x_m\ \} = e\ ;\ b : R}$$

Struct pattern syntax is specified in “Match Expressions and Patterns”. Contextual forwarding fields are specified in “Scopes and Bindings”.

## 3.9 Enum Types

**Enum declaration well-formedness.** An enum declaration is well-formed when every variant-local type binder kind is well-formed and every declared variant payload type is well-formed under the collected type-constructor context, the enum's type parameters, and the variant-local hidden type binders.

```
enum E[A1, ..., An] {
  v1[B11, ..., B1r1](P11, ..., P1m1)
  ...
  vj[Bj1, ..., Bjrj](Pj1, ..., Pjmj)
} // enum definition
```

$$\frac{\begin{cases} E \to \mathcal{E}\left(A_1, ..., A_n; v_j\left[B_{j,1}:K_{j,1},...,B_{j,r_j}:K_{j,r_j}\right]\left(P_{j,1}, ..., P_{j,m_j}\right)\right) \in \Delta \\ \forall j, p.\Delta A_1, ..., A_n \vdash K_{j,p}\checkmark \\ \forall j, k.\Delta A_1, ..., A_n, B_{j,1} : K_{j,1}, ..., B_{j,r_j} : K_{j,r_j} \vdash P_{j,k} : \text{Type} \end{cases}}{\Delta \vdash \text{enum } E\ [\ A_1, ..., A_n\ ]\ \{\ v_j\ [\ B_{j,1}, ..., B_{j,r_j}\ ]\ (P_j)\ \}\ \checkmark}$$

**Variant constructor introduction.** Each declared variant constructor is an introduction form for values of its owning enum type. A variant constructor does not introduce a separate variant type. The payload expressions must match the declared variant payload types after substituting the enum type arguments.

```
E[T1, ..., Tn]::v(e1, ..., em) // qualified variant constructor call
E[T1, ..., Tn]::v[U1, ..., Ur](e1, ..., em) // constructor call with hidden witnesses
v(e1, ..., em) // unqualified constructor call after unambiguous resolution
```

For an enum declaration shape $D$, $D \vdash v : (B_1 : K_1, ..., B_r : K_r; P_1, ..., P_m)$ states that $D$ declares variant $v$ with hidden type binders and payload type sequence $(P_1, ..., P_m)$.

The judgment $\Delta\Theta \vdash E[T_1, ..., T_n] :: v[U_1, ..., U_r] \to (Q_1, ..., Q_m)$ instantiates that payload sequence at the enum type arguments and the chosen hidden witnesses.

$$\frac{\begin{cases} E \to D \in \Delta \\ D \vdash v : (B_1 : K_1, ..., B_r : K_r; P_1, ..., P_m) \\ \pi(D) = (A_1, ..., A_n) \\ \sigma = [T_1/A_1, ..., T_n/A_n] \\ \forall i.\Delta\Theta \vdash T_i : \text{Type} \\ \forall p.\Delta\Theta \vdash U_p : K_p\sigma \\ \rho = [U_1/B_1, ..., U_r/B_r] \end{cases}}{\Delta\Theta \vdash E[T_1, ..., T_n] :: v[U_1, ..., U_r] \to (P_1\sigma\rho, ..., P_m\sigma\rho)}$$

$$\frac{\begin{cases} \Delta\Theta \vdash E[T_1, ..., T_n] :: v[U_1, ..., U_r] \to (Q_1, ..., Q_m) \\ \forall k.\Gamma \vdash e_k : Q_k \end{cases}}{\Gamma \vdash E\ [\ T_1, ..., T_n\ ] :: v\ [\ U_1, ..., U_r\ ]\ (e_1, ..., e_m) : E[T_1, ..., T_n]}$$

An unqualified enum variant expression is typed by the same rule after name resolution has selected exactly one visible enum variant. Variant-local hidden witnesses may be written explicitly after the variant name or selected by local type inference when there is exactly one solution.

**Enum match elimination.** A match expression eliminates an enum value. For an enum scrutinee type, every declared variant must be covered.

```
match e {
  E::v1[B11, ..., B1r1](x11, ..., x1m1) => b1
  ...
  E::vq[Bq1, ..., Bqrq](xq1, ..., xqmq) => bq
}
```

$$\frac{\begin{cases} E \to D \in \Delta \\ \pi(D) = (A_1, ..., A_n) \\ D \vdash v : (B_1 : K_1, ..., B_r : K_r; P_1, ..., P_m) \\ \sigma = [T_1/A_1, ..., T_n/A_n] \\ \Gamma, B_1 : K_1\sigma, ..., B_r : K_r\sigma, x_1 : P_1\sigma, ..., x_m : P_m\sigma \vdash b : R \\ N(B_1, ..., B_r; R) \end{cases}}{\Gamma \vdash E :: v\ [\ B_1, ..., B_r\ ]\ (x_1, ..., x_m) \to b : \alpha(E[T_1, ..., T_n], v, R)}$$

$$\frac{\begin{cases} \Gamma \vdash e : E[T_1, ..., T_n] \\ E \to D \in \Delta \\ D \vdash \nu(v_1, ..., v_q) \\ \forall j . \Gamma \vdash a_j : \alpha(E[T_1, ..., T_n], v_j, R) \end{cases}}{\Gamma \vdash \text{match } e\ \{\ a_1, ..., a_q\ \} : R}$$

Pattern syntax and exhaustiveness diagnostics are specified in “Match Expressions and Patterns”.

# 4 Type Inference

Lane2 permits local type inference only at syntactic positions where the omitted type is determined locally.

A binding’s type is not determined from later uses of the bound name.

Local `let` bindings may omit type annotations only when the initializer has a type without using later references to the bound name.

Function literals may omit parameter types only when the immediately surrounding context provides a function type. Otherwise, every value parameter of a function literal must have an explicit type annotation.

Generic function literals without an immediately surrounding generic function type must explicitly declare type parameters and explicitly annotate value parameters.

Generic function calls and generic data constructors may instantiate type parameters at the use site when the use site is unambiguous.

# 5 Builtins

`builtin("...")` is an unsafe expression.

The checker does not interpret intrinsic strings. Instead, `builtin` receives its type from direct expected context:

```
fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}
```

This is valid because the function return type provides the expected type `Int` for the body expression.

This is invalid:

```
let x = builtin("%anything")
```

There is no direct expected type.

Incorrect builtin use can produce undefined behavior. Lane2's type safety guarantee applies only to programs that do not misuse `builtin`.

### 5.1 Required Intrinsics

A conforming Lane2/Core v1 implementation provides these portable intrinsic names:

| Intrinsic | Expected type |
|---|---|
| `%i64_add` | `(Int, Int) -> Int` |
| `%i64_sub` | `(Int, Int) -> Int` |
| `%i64_mul` | `(Int, Int) -> Int` |
| `%i64_div` | `(Int, Int) -> Int` |
| `%i64_rem` | `(Int, Int) -> Int` |
| `%i64_neg` | `(Int) -> Int` |
| `%i64_equal` | `(Int, Int) -> Bool` |
| `%i64_less` | `(Int, Int) -> Bool` |
| `%string_equal` | `(String, String) -> Bool` |

Table 2: Required intrinsic names and expected types

Other intrinsic names are implementation-defined unsafe builtins.

`%bool_and`, `%bool_or`, `%bool_not`, and `%bool_equal` are not required intrinsics. The standard prelude defines boolean operations as ordinary Lane2 functions and offered operation values using `if`; `&&` and `||` supply their right operands as thunks.

## 6 Prelude

The standard prelude is implementation-supplied Lane2 source checked before user code. It is not a module system.

Prelude declarations provide standard operation structs, primitive wrappers around required intrinsics, derived primitive operations written in Lane2, and prelude-provided contextual offers.

Prelude entries are ordinary Lane2 values and types. Operation structs are API conventions that group short operation fields such as `add`, `equal`, and `less`.

Operation laws are API conventions, not compiler-checked rules.

### 6.1 Standard Prelude Source

The v1 standard prelude source is stored in `lane-std/prelude.lane` and rendered here:

```
// Lane2 v1 standard prelude.
// This file is the source of truth for prelude declarations shown in the
// language specification.

struct Add[T] {
  add : (T, T) -> T
}

struct Sub[T] {
  sub : (T, T) -> T
}

struct Mul[T] {
  mul : (T, T) -> T
}

struct Div[T] {
  div : (T, T) -> T
```

```
}

struct Rem[T] {
  rem : (T, T) -> T
}

struct Neg[T] {
  neg : (T) -> T
}

struct And {
  and : (Bool, () -> Bool) -> Bool
}

struct Or {
  or : (Bool, () -> Bool) -> Bool
}

struct Not {
  not : (Bool) -> Bool
}

struct Equal[T] {
  equal : (T, T) -> Bool
  not_equal : (T, T) -> Bool
}

struct Compare[T] {
  offer equal_impl : Equal[T]
  less : (T, T) -> Bool
  less_eq : (T, T) -> Bool
  greater : (T, T) -> Bool
  greater_eq : (T, T) -> Bool
}

fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}

fn[T] op_sub(a : T, b : T, auto op : Sub[T]) -> T {
  op.sub(a, b)
}

fn[T] op_mul(a : T, b : T, auto op : Mul[T]) -> T {
  op.mul(a, b)
}

fn[T] op_div(a : T, b : T, auto op : Div[T]) -> T {
  op.div(a, b)
}

fn[T] op_rem(a : T, b : T, auto op : Rem[T]) -> T {
  op.rem(a, b)
}

fn[T] op_neg(value : T, auto op : Neg[T]) -> T {
  op.neg(value)
}

fn op_and(a : Bool, b : () -> Bool, auto op : And) -> Bool {
  op.and(a, b)
```

```
}

fn op_or(a : Bool, b : () -> Bool, auto op : Or) -> Bool {
  op.or(a, b)
}

fn op_not(value : Bool, auto op : Not) -> Bool {
  op.not(value)
}

fn[T] op_equal(a : T, b : T, auto op : Equal[T]) -> Bool {
  op.equal(a, b)
}

fn[T] op_not_equal(a : T, b : T, auto op : Equal[T]) -> Bool {
  op.not_equal(a, b)
}

fn[T] op_less(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.less(a, b)
}

fn[T] op_less_eq(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.less_eq(a, b)
}

fn[T] op_greater(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.greater(a, b)
}

fn[T] op_greater_eq(a : T, b : T, auto op : Compare[T]) -> Bool {
  op.greater_eq(a, b)
}

fn int_add(a : Int, b : Int) -> Int {
  builtin("%i64_add")
}

fn int_sub(a : Int, b : Int) -> Int {
  builtin("%i64_sub")
}

fn int_mul(a : Int, b : Int) -> Int {
  builtin("%i64_mul")
}

fn int_div(a : Int, b : Int) -> Int {
  builtin("%i64_div")
}

fn int_rem(a : Int, b : Int) -> Int {
  builtin("%i64_rem")
}

fn int_neg(a : Int) -> Int {
  builtin("%i64_neg")
}

fn int_equal(a : Int, b : Int) -> Bool {
  builtin("%i64_equal")
}
```

```
fn int_less(a : Int, b : Int) -> Bool {
  builtin("%i64_less")
}

fn string_equal(a : String, b : String) -> Bool {
  builtin("%string_equal")
}

fn bool_and(a : Bool, b : () -> Bool) -> Bool {
  if a {
    b()
  } else {
    false
  }
}

fn bool_or(a : Bool, b : () -> Bool) -> Bool {
  if a {
    true
  } else {
    b()
  }
}

fn bool_not(a : Bool) -> Bool {
  if a {
    false
  } else {
    true
  }
}

fn bool_equal(a : Bool, b : Bool) -> Bool {
  if a {
    b
  } else {
    bool_not(b)
  }
}

fn[T] make_equal(equal : (T, T) -> Bool) -> Equal[T] {
  Equal::{
    equal,
    not_equal: fn(a : T, b : T) -> Bool {
      if equal(a, b) {
        false
      } else {
        true
      }
    },
  }
}

fn[T] make_compare(
  equal_impl : Equal[T],
  less : (T, T) -> Bool
) -> Compare[T] {
  Compare::{
    equal_impl,
    less,
```

```
    less_eq: fn(a : T, b : T) -> Bool {
      if equal_impl.equal(a, b) {
        true
      } else {
        less(a, b)
      }
    },
    greater: fn(a : T, b : T) -> Bool {
      less(b, a)
    },
    greater_eq: fn(a : T, b : T) -> Bool {
      if equal_impl.equal(a, b) {
        true
      } else {
        less(b, a)
      }
    },
  }
}

offer int_add_ops : Add[Int] = Add::{ add: int_add }
offer int_sub_ops : Sub[Int] = Sub::{ sub: int_sub }
offer int_mul_ops : Mul[Int] = Mul::{ mul: int_mul }
offer int_div_ops : Div[Int] = Div::{ div: int_div }
offer int_rem_ops : Rem[Int] = Rem::{ rem: int_rem }
offer int_neg_ops : Neg[Int] = Neg::{ neg: int_neg }

let int_equal_ops : Equal[Int] = make_equal(int_equal)
offer int_compare_ops : Compare[Int] = make_compare(int_equal_ops, int_less)

offer bool_and_ops : And = And::{ and: bool_and }
offer bool_or_ops : Or = Or::{ or: bool_or }
offer bool_not_ops : Not = Not::{ not: bool_not }
offer bool_equal_ops : Equal[Bool] = make_equal(bool_equal)

offer string_equal_ops : Equal[String] = make_equal(string_equal)
```

Boolean conjunction, disjunction, negation, and equality do not require boolean intrinsics; they can be defined with ordinary `if` expressions in the prelude.

# 7 Declarations

Declarations introduce types, functions, values, and contextual offers.

## 7.1 Top-Level Declarations

Top-level declarations are described by the `topLevelDefinition` grammar in “Syntax and Grammar”.

Top-level forms include struct declarations, enum declarations, named function declarations, typed value declarations, and offered value definitions.

## 7.2 Struct Declarations

Struct declarations use the grammar in “Syntax and Grammar”. Their nominal type rules are specified in “Type System”.

## 7.3 Enum Declarations

Enum declarations use the grammar in “Syntax and Grammar”. Their nominal type rules are specified in “Type System”. Variant names may be lowercase or uppercase; capitalization has no semantic role.

## 7.4 Function Declarations

Functions are uncurried. There is no automatic currying or partial application.

Function definitions use block bodies:

```
fn add(a : Int, b : Int) -> Int {
  a + b
}
```

There is no `= expr` function body form.

Function result types use `->`, not `:`.

All named functions must state every parameter type and the result type.

Generic named functions place type parameters after `fn` and before the name:

```
fn[A] id(value : A) -> A {
  value
}
```

Function parameters are positional in v1. Labeled function parameters are not supported.

Named function definitions may mark a trailing suffix of parameters with `auto`. These contextual parameters remain ordinary parameters in the function type, but a direct named call may omit them and let Contextual Resolution supply them.

```
fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}
```

Contextual parameters must appear as a contiguous suffix of the parameter list. Function literals cannot declare `auto` parameters.

A parameter may be marked `offer`; it is then added to the function body's contextual offer environment. The `offer` modifier does not affect the function type or call syntax. Function literals may use `offer` parameters.

```
fn[T] twice(x : T, auto offer add : Add[T]) -> T {
  op_add(x, x)
}
```

Function literals produce function values:

```
let f : (Int, Int) -> Int = fn(a, b) {
  a + b
}
```

Generic function literals place type parameters after `fn`:

```
let id = fn[A](value : A) {
  value
}
```

## 7.5 Let Declarations

Named top-level `let` declarations must include type annotations.

Local `let` declarations may omit type annotations when their initializer can synthesize a type by local type inference.

An offered value definition has the form `offer name : Type = expression`. It defines a named value and immediately adds that value to the contextual offer environment.

Offered value definitions must include type annotations:

```
offer int_add_ops : Add[Int] = Add::{ add: int_add }
```

Local offered value definitions use the same annotated form:

```
{
  offer ops : Add[Int] = Add::{ add: int_add }
  1 + 2
}
```

Offered value definitions must be named. They are not forward-visible and do not make the defined value available as an offer inside its own initializer.

# 8 Scopes and Bindings

## 8.1 Notations

| Notation | Meaning |
|---|---|
| $R$ | resolution environment |
| $\Delta$ | type-name namespace |
| $\Gamma$ | ordinary value-binding scope stack |
| $\Omega$ | contextual offer scope stack |
| $s_T, s_V, s_F, s_R$ | type, value, field, and variant symbols |
| $x$ | ordinary value name |
| $C$ | type name |
| $S$ | struct type constructor |
| $E$ | enum type constructor |
| $B$ | existential type binder |
| $l$ | struct field name |
| $v$ | enum variant name |
| $A$ | contextual parameter target type |
| $R = (\Delta, \Gamma, \Omega)$ | resolution environment components |
| $\Delta \vdash C \to s_T$ | type-name resolution |
| $\Gamma \vdash x \to s_V$ | ordinary value resolution |
| $\Omega \vdash A \to s_V$ | contextual resolution by type |
| $R \vdash d \to R'$ | declaration resolution |
| $R \vdash e \to e'$ | expression-name resolution |
| $\tau(s_V) = A$ | known type of a value symbol |
| $\varphi(s_V)$ | contextual offers forwarded from a struct value |
| $\nu(s_T, v) = s_R$ | variant owned by an enum type symbol |

Table 3: Scope and binding notations

## 8.2 Resolution Model

Name resolution maps source names to symbols before Buslane Core Language is produced. It does not evaluate expressions. Contextual Resolution is a later type-directed step that supplies omitted contextual arguments from visible offers after ordinary call inference has determined the target contextual parameter types.

The resolution environment has separate namespaces for type names, value names, and contextual offers. Field and variant symbols are owned by nominal type symbols and are not global value bindings.

```
// source names
TypeName
value_name
value_name.field_name
TypeName::variant_name
```

$$R = (\Delta, \Gamma, \Omega)$$

$$\Delta \vdash C \to s_T$$

$$\Gamma \vdash x \to s_V$$

$$\Omega \vdash A \to s_V$$

## 8.3 Namespaces

Type and value names are resolved by different lookup judgments. The same source spelling may therefore be bound in both namespaces without conflict.

```
enum Option[A] {
  none
  some(A)
}

let Option : Int = 42
```

**Type-name lookup.** A type name resolves through the type namespace.

$$\frac{C \to s_T \in \Delta}{\Delta \vdash C \to s_T}$$

**Ordinary value lookup.** A value name resolves through the ordinary value-binding scope stack.

$$\frac{x \to s_V \in \Gamma}{\Gamma \vdash x \to s_V}$$

Ordinary value lookup never searches the contextual offer environment.

Qualified type-member syntax resolves its left-hand side in the type namespace.

```
Option::some(1) // Option is resolved as a type name
```

$$\frac{\Delta \vdash E \to s_T \quad \nu(s_T, v) = s_R}{R \vdash E :: v \to s_R}$$

## 8.4 Top-Level Scope

Top-level `struct`, `enum`, and `fn` declarations are collected before top-level value initializers are resolved. Top-level custom types and functions may therefore refer to each other regardless of textual order.

```
struct S { value : T }
enum T { wrap(S) }
fn f(x : T) -> T { x }
```

$$\frac{\mathcal{T}(d_1, ..., d_n) = \Delta \quad \mathcal{F}(d_1, ..., d_n) = \Gamma}{\mathcal{R}(d_1, ..., d_n) \to (\Delta, \Gamma)}$$

Top-level value definitions and top-level offered value definitions are resolved in source order. A top-level initializer may refer only to values and contextual offers available at that declaration point. A top-level function body is resolved after the complete top-level function, value, and contextual offer environments are known.

```
let x : Int = earlier
offer y_ops : Add[Int] = Add::{ add: int_add }
```

$$\frac{R_i \ ; \ K_i \vdash e_i \to e_{i'} \quad R_{i+1} = R_i, x_i \to s_i}{R_i \vdash \text{let } x_i : T_i = e_i \to R_{i+1}}$$

An offered value definition checks its initializer with the current environment, defines the named value, and offers that value from the declaration point forward.

```
offer ops : Add[Int] = Add::{ add: int_add }
```

$$\frac{R_i \ ; \ K_i \vdash e \to e' \quad R_{i+1} = R_i, x \to s_V, \omega(s_V)}{R_i \vdash \text{offer } x : T = e \to R_{i+1}}$$

Top-level value initializers are checked only with contextual offers available earlier in source order. Top-level function bodies are checked with the complete top-level contextual offer environment.

## 8.5 Local Scope

Local `let`, local named `fn`, and local offered value definitions are sequential. A local binding is visible only to later local items and the final expression in the same block.

```
{
  let x = e
  fn f(y : T) -> U { b }
  offer ops : Add[Int] = Add::{ add: int_add }
  result
}
```

$$\frac{R_i \ ; \ K_i \vdash e_i \to e_{i'} \quad R_{i+1} = R_i, x_i \to s_i}{R_i \vdash \text{let } x_i = e_i \to R_{i+1}}$$

A local pattern `let` is sequential and may introduce both value binders and type binders. It is the source form used to open existential structs over the remainder of the block.

```
{
  let Hide::{ T, val } = h
  body
}
```

$$\frac{\begin{cases} R_i \ ; \ K_i \vdash e_i \to e_{i'} \\ \beta(p, e_{i'}) = (B_1, ..., B_r; x_1 \to s_1, ..., x_m \to s_m) \\ R_{i+1} = R_i, B_1, ..., B_r, x_1 \to s_1, ..., x_m \to s_m \end{cases}}{R_i \vdash \text{let } p = e_i \to R_{i+1}}$$

$$\frac{R_{i+1} = R_i, f \to s_f \quad R_{i+1} \ ; \ K \vdash b \to b'}{R_i \vdash \text{fn } f : U \ \{ \ b \ \} \to R_{i+1}}$$

Local value names may shadow earlier value names from outer layers. Ordinary value bindings in the same layer must have distinct names.

```
let x = outer
{
  let x = inner
  x
}
```

$$\frac{\Gamma = \Gamma_0, (x \to s_1) \quad \Gamma' = \Gamma, (x \to s_2)}{\Gamma' \vdash x \to s_2}$$

## 8.6 Contextual Offers

An offered value definition introduces a named value with an explicit type annotation and contributes that value to Contextual Resolution.

```
offer int_add_ops : Add[Int] = Add::{ add: int_add }
```

$$\frac{\Gamma \vdash x \to s_V \quad \tau(s_V) = A}{(\Delta, \Gamma, \Omega) \vdash \text{offer } x \to (A \to s_V), \varphi(s_V)}$$

A contextual offer affects only Contextual Resolution. It does not expose fields as unqualified names.

```
offer int_add_ops : Add[Int] = Add::{ add: int_add }
1 + 2
```

Multiple visible offers may have the same type. This is not an error at the offered value definition point. Ambiguity is reported only when Contextual Resolution needs a unique value of that type.

```
offer left_add : Add[Int] = Add::{ add: int_add }
offer right_add : Add[Int] = Add::{ add: alternate_int_add }
1 + 2 // ambiguous if both offers have type Add[Int]
```

$$\frac{\Omega \vdash A \to (s_1, ..., s_n) \quad n = 1}{\Omega \vdash \omega(A) \to s_1}$$

$$\frac{\Omega \vdash A \to ()}{\Omega \vdash \omega(A) \to \varepsilon_m}$$

$$\frac{\Omega \vdash A \to (s_1, ..., s_n) \quad n > 1}{\Omega \vdash \omega(A) \to \varepsilon_a}$$

Repeating the same offer identity is semantically idempotent but should produce a warning. Contextual offer deduplication uses offer identity, not runtime value equality.

Visible contextual offers from nested lexical scopes are combined rather than shadowed. Nested local function bodies can see contextual offers from their lexical environment.

## 8.7 Contextual Forwarding Fields

When a value is offered, fields declared with the `offer` modifier are offered recursively. Forwarding contributes only to Contextual Resolution and never to ordinary value lookup.

```
struct Compare[T] {
  offer equal_impl : Equal[T]
  less : (T, T) -> Bool
}

offer int_compare_ops : Compare[Int] = Compare::{ equal_impl, less: int_less }
```

Offering `int_compare_ops : Compare[Int]` also offers `int_compare_ops.equal_impl : Equal[Int]`. Forwarding uses normal nominal field typing of the containing value. Recursive forwarding must detect cycles.

$$\frac{\tau(s_V) = S[T_1, ..., T_n] \quad \varphi_{S(S[T_1,...,T_n])} = (l_1 : A_1, ..., l_m : A_m)}{\varphi(s_V) = (A_1 \to s_V.l_1, ..., A_m \to s_V.l_m), \varphi(s_V.l_1), ..., \varphi(s_V.l_m)}$$

## 8.8 Direct Named Calls and Contextual Arguments

A function introduced by a named function definition may declare contextual parameters with `auto`. They must form a contiguous suffix of the parameter list. A contextual parameter remains an ordinary parameter in the function type.

```
fn[T] op_add(a : T, b : T, auto op : Add[T]) -> T {
  op.add(a, b)
}
```

A direct named function call is a call whose callee is a direct value reference to a function definition symbol. Only direct named function calls may omit contextual parameters or use explicit contextual arguments. Calls through ordinary function values must provide every argument positionally according to the function type.

```
op_add(1, 2)
op_add(1, 2, op=custom_add)
```

Explicit contextual arguments are named. Positional call arguments cannot fill contextual parameters.

```
op_add(1, 2, custom_add) // invalid
```

The right-hand side of an explicit contextual argument is an ordinary expression. Explicit contextual arguments participate in ordinary generic call inference before Contextual Resolution supplies the remaining omitted contextual parameters.

Contextual Resolution never infers generic type arguments for the call being completed. It runs only after ordinary positional arguments, explicit contextual arguments, and the direct expected result type have determined the target contextual parameter types.

### 8.9 Special Resolution Forms

An unqualified enum variant expression may resolve without qualification only when exactly one visible enum variant has that name.

```
some(1)
```

$$\frac{\nu_{v(\Delta,v)} = (s_R)}{R \ ; \ K \vdash v \to s_R}$$

Field access first resolves and checks its base as a unique value, then selects a field owned by the base struct type.

```
ops.op_add
```

$$\frac{R \ ; \ K \vdash x \to s_V \quad \tau(s_V) = S[T_1, ..., T_n] \quad \mu(S, l) = s_F}{R \ ; \ K \vdash x.l \to s_F}$$

Operator aliases first map to fixed ordinary operation names, then elaborate to direct named calls when the operation name resolves to a function definition symbol. `&&` and `||` additionally thunk the right operand before the call is checked.

```
a + b  // elaborates to op_add(a, b)
a && b // elaborates to op_and(a, fn() { b })
```

$$\frac{\eta(o) = x \quad \Gamma \vdash x \to s_V}{R \ ; \ K \vdash o \to s_V}$$

# 9 Expressions

Expressions compute values. Lane2 v1 is expression-oriented and has no expression statements.

This chapter specifies source expression elaboration into Buslane. The appropriate semantic object here is an elaboration judgment, not a denotational semantics: source expressions contain conveniences that do not exist in Buslane, including `if`, field access, pipeline, source struct syntax, source enum syntax, operators, contextual arguments, and unsafe builtin syntax. Evaluation is specified only for the resulting Buslane expression in “Buslane Core Language”.

## 9.1 Notations

| | |
|---|---|
| $L$ | elaboration environment |
| $M$ | Buslane metadata registry produced by declarations |
| $\Theta$ | visible type binders |
| $\Gamma$ | visible value binders |
| $e, a, p$ | source expression, source match arm, and source pattern |
| $b, c, h$ | Buslane expression |
| $T, U, R$ | Buslane type |
| $l$ | source literal or source field name |
| $o$ | source operator token |
| $K$ | Buslane data-constructor identity |
| $x, y, f$ | Buslane value identity |
| $L; \Theta\Gamma \vdash e \to b : T$ | source expression elaboration |
| $L; \Theta\Gamma \vdash a \to A : \alpha(x, T, R)$ | source match-arm elaboration |
| $L \vdash I_1, ..., I_n; e \to b : T$ | block-item sequence elaboration |
| $I_{L(x)} = y$ | source value symbol $x$ maps to Buslane value identity $y$ |
| $F_{L(S,l)} = f$ | source field selector for struct $S$ and field $l$ |
| $K_{L(S)} = K$ | Buslane data constructor generated for source struct $S$ |
| $K_{L(E,v)} = K$ | Buslane data constructor generated for source enum variant $E :: v$ |
| $O_{L(o)} = f$ | source operator token maps to an operation function identity |
| $X_{L(s,T)} = f$ | builtin string $s$ at expected type $T$ maps to an external value identity |

Table 4: Expression elaboration notations

The elaboration relation is defined after parsing, name resolution, local type inference, and contextual resolution. Therefore every rule below receives resolved value identities, selected type arguments, selected contextual arguments, and checked result types as inputs. Elaboration does not perform overload search or evaluation.

## 9.2 Blocks

A block expression contains local items followed by one final expression.

```
{
  let x = 1
  fn double(n : Int) -> Int {
    n + n
  }
  double(x)
}
```

Allowed local items:

- `let`,
- named `fn`,
- `offer`.

Local type definitions are not supported in v1.

A block does not contain expression statements. This is invalid:

```
{
  f(x)
  g(y)
}
```

An empty block is invalid. Write `()` explicitly when a `Unit` value is required.

Block elaboration turns local items into nested Buslane binders. Contextual offers affect elaboration of later source expressions, but they do not appear as Buslane operations.

$$\frac{L; \Theta\Gamma \vdash e \to b : T}{L \vdash; e \to b : T} \quad \text{(E-BlockDone)}$$

$$\frac{\begin{cases} L; \Theta\Gamma \vdash e_1 \to b_1 : T_1 \\ L \vdash I_2, ..., I_n; e \to b_2 : T_2 \end{cases}}{L \vdash \text{let } x = e_1, I_2, ..., I_n; e \to \text{let } x = b_1 \text{ in } b_2 : T_2} \quad \text{(E-BlockLet)}$$

$$\frac{\begin{cases} L; \Theta\Gamma \vdash e_1 \to b_1 : T_1 \\ L, \omega(x : T_1) \vdash I_2, ..., I_n; e \to b_2 : T_2 \end{cases}}{L \vdash \text{offer } x : T_1 = e_1, I_2, ..., I_n; e \to \text{let } x = b_1 \text{ in } b_2 : T_2} \quad \text{(E-BlockOffer)}$$

$$\frac{\begin{cases} L; \Theta\Gamma, f \vdash h \to b_1 : T_1 \\ L \vdash I_2, ..., I_n; e \to b_2 : T_2 \end{cases}}{L \vdash \text{fn } f = h, I_2, ..., I_n; e \to \text{letrec } (f = b_1) \text{ in } b_2 : T_2} \quad \text{(E-BlockFn)}$$

## 9.3 Conditional Expressions

`if` is an expression:

```
if condition {
  then_value
} else {
  else_value
}
```

The `else` branch is mandatory.

Both branches must have the same type.

`if` elaborates to a Buslane match over `Bool`.

$$\frac{\begin{cases} L; \Theta\Gamma \vdash c \to b_c : \text{Bool} \\ L; \Theta\Gamma \vdash e_1 \to b_1 : T \\ L; \Theta\Gamma \vdash e_2 \to b_2 : T \\ M \vdash x_c : \text{Bool} \end{cases}}{L; \Theta\Gamma \vdash \text{if } c \text{ then } e_1 \text{ else } e_2 \to \text{match } b_c \text{ as } x_c \to T \ \{ \text{ true} \to b_1, \text{false} \to b_2 \ \} : T} \quad \text{(E-If)}$$

## 9.4 Calls, Field Access, and Pipeline

Function call:

```
f(x, y)
```

A direct named function call may omit trailing contextual parameters declared with `auto`. The omitted parameters are supplied by Contextual Resolution after ordinary argument checking and generic call inference determine their target types.

```
op_add(1, 2)
```

A caller may explicitly supply contextual parameters by name:

```
op_add(1, 2, op=custom_add)
```

Named call arguments are valid only for contextual parameters of direct named function calls. Non-contextual parameters cannot be supplied by name, and contextual parameters cannot be supplied positionally.

Field access:

```
ops.add
```

The base of field access must resolve and check as a unique value before field selection.

Field access followed by call:

```
ops.add(x, y)
```

This is not method call syntax. Lane2 v1 has no method receiver lookup.

Pipeline syntax is supported:

```
value |> f(a, b)
```

It rewrites to:

```
f(value, a, b)
```

The right-hand side of `|>` must be a call or function literal.

The following are invalid:

```
value |> f
value |> f(_, y)
```

Placeholders are not supported.

Resolved value references elaborate to Buslane value identities.

$$\frac{I_{L(x)} = y \quad M \vdash y : T}{L; \Theta\Gamma \vdash x \to y : T} \text{(E-Value)}$$

Function calls elaborate to Buslane calls after local type inference and contextual resolution have supplied all type arguments and value arguments.

$$\frac{\begin{cases} L; \Theta\Gamma \vdash f \to b_f : (T_1, ..., T_n) \to R \\ \forall i. L; \Theta\Gamma \vdash e_i \to b_i : T_i \end{cases}}{L; \Theta\Gamma \vdash f(e_1, ..., e_n) \to b_{f(b_1, ..., b_n)} : R} \text{(E-Call)}$$

$$\frac{\begin{cases} L; \Theta\Gamma \vdash f \to b_f : \forall A_1, ..., A_n.T \\ \forall i. M; \Theta \vdash U_i : \text{Type} \end{cases}}{L; \Theta\Gamma \vdash f[U_1, ..., U_n] \to b_{f[U_1, ..., U_n]} : T[U_1/A_1, ..., U_n/A_n]} \text{(E-TypeCall)}$$

Field access elaborates to a generated selector function call. Buslane has no field-access expression.

$$\frac{\begin{cases} L; \Theta\Gamma \vdash e \to b : S[T_1, ..., T_n] \\ F_{L(S,l)} = f \\ M \vdash f : (S[T_1, ..., T_n]) \to R \end{cases}}{L; \Theta\Gamma \vdash e.l \to f(b) : R} \quad \text{(E-Field)}$$

Pipeline is elaborated by inserting the left expression as the first positional argument of the right call form.

$$\frac{L; \Theta\Gamma \vdash f(e_0, e_1, ..., e_n) \to b : T}{L; \Theta\Gamma \vdash e_0 \ |> \ f(e_1, ..., e_n) \to b : T} \quad \text{(E-Pipeline)}$$

### 9.5 Struct and Enum Expressions

Struct literals are qualified:

```
let p : Point = Point::{ x: 1, y: 2 }
```

Struct field punning is allowed:

```
let p : Point = Point::{ x, y }
```

This is equivalent to:

```
let p : Point = Point::{ x: x, y: y }
```

Struct literals must provide every value field exactly once. Structs with hidden type members must also provide every type-member witness.

```
let h : Hide = Hide::{ T = Int, val: 5 }
```

Type-member witnesses use `=`. Value fields use `:`.

The following struct expression forms are not supported:

- anonymous record literals,
- default fields,
- spread,
- copy update,
- field update.

Fields are accessed with dot syntax:

```
p.x
```

V1 has no field visibility modifiers. Every struct field is accessible wherever the struct value is accessible.

If a field type mentions an unopened hidden type member, field access is invalid until a struct pattern has opened that hidden type.

Enum variants are scoped to their enum. Payloadless variants are values and are written without call parentheses:

```
let x : Option[Int] = Option::none
```

This is invalid:

```
Option::none()
```

Variant payloads are positional:

```
enum Expr {
  int(Int)
  add(Expr, Expr)
}
```

Labeled variant payloads are not part of v1. Named product data must be represented with a struct.

An enum variant may declare hidden type binders that are chosen at construction:

```
enum Hide {
  hide[T](T)
}

let h : Hide = Hide::hide[Int](5)
```

In expressions, unqualified variants are allowed when unambiguous:

```
let x : Option[Int] = some(1)
```

If more than one visible enum has a variant named `some`, the unqualified expression is invalid unless another directly local rule disambiguates it. A qualified variant is always allowed:

```
let x : Option[Int] = Option::some(1)
```

Source struct literals elaborate to calls of the generated Buslane data constructor for the struct. Source field order is erased; Buslane receives payloads in declaration order.

$$\frac{\begin{cases} p = S ::\{\ B_1 = U_1, ..., B_r = U_r, l_1 : e_1, ..., l_m : e_m\ \} \\ K_{L(S)} = K \\ \sigma_{S(S[T_1,...,T_n])} = (B_1 : K_1, ..., B_r : K_r; l_1 : Q_1, ..., l_m : Q_m) \\ \forall i. L; \Theta\Gamma \vdash e_i \to b_i : Q_{i[U_1/B_1,...,U_r/B_r]} \\ c = K[T_1, ..., T_n; U_1, ..., U_r](b_1, ..., b_m) \end{cases}}{L; \Theta\Gamma \vdash p \to c : S[T_1, ..., T_n]} \quad \text{(E-Struct)}$$

Source enum variant expressions elaborate to calls of the selected Buslane data constructor. Qualified and unqualified source forms use the same rule after name resolution selects the variant.

$$\frac{\begin{cases} p = E :: v\ [\ U_1, ..., U_r\ ]\ (e_1, ..., e_m) \\ K_{L(E,v)} = K \\ \Delta\Theta \vdash E[T_1, ..., T_n] :: v[U_1, ..., U_r] \to (Q_1, ..., Q_m) \\ \forall i. L; \Theta\Gamma \vdash e_i \to b_i : Q_i \\ c = K[T_1, ..., T_n; U_1, ..., U_r](b_1, ..., b_m) \end{cases}}{L; \Theta\Gamma \vdash p \to c : E[T_1, ..., T_n]} \quad \text{(E-Variant)}$$

## 9.6 Match Expressions and Patterns

`match` is an expression:

```
match value {
  Option::none => fallback
  Option::some(x) => x
}
```

Match arms use `pattern => expression`.

Every match must be exhaustive.

V1 patterns:

- wildcard pattern: _,
- variable binding pattern: `name`,
- literal pattern,
- qualified enum variant pattern, optionally with hidden type binders,
- qualified struct pattern.

Enum variants in patterns must be qualified:

```
Option::some(x)
Option::none
Hide::hide[T](val)
```

When a variant declares hidden type binders, the variant pattern opens fresh abstract type binders for the arm body. The arm result type must not mention those binders.

Bare identifiers in patterns are always variable bindings:

```
match value {
  x => x
}
```

Struct patterns use qualified syntax and must list all fields:

```
match p {
  Point::{ x, y } => x + y
}
```

Struct pattern punning is allowed. Explicit renaming is allowed:

```
match p {
  Point::{ x: a, y: b } => a + b
}
```

Struct patterns for structs with type members list type-member binders before value fields. A type-member binder introduces an abstract type for the remaining local scope or match arm. _ ignores one type member and does not introduce a usable type name.

```
let Hide::{ T, val } = h
let Hide::{ _, val } = h
```

Hidden type binders opened by enum or struct patterns are scoped to the pattern body. They must not occur free in the type of the body result unless the value is repacked into another existential before leaving that body.

Rest patterns, spread patterns, guards, or-patterns, as-patterns, and `is` pattern expressions are not supported in v1.

Source match elaborates to Buslane match. Every source pattern is lowered to one Buslane alternative form. Struct patterns are data-constructor alternatives for the generated struct constructor.

$$\frac{\begin{cases} L; \Theta\Gamma \vdash e \to b : T \\ M \vdash x_m : T \\ \forall i. L; \Theta\Gamma \vdash a_i \to A_i : \alpha(x_m, T, R) \end{cases}}{L; \Theta\Gamma \vdash \text{match } e \ \{\ a_1, ..., a_n\ \} \to \text{match } b \text{ as } x_m \to R \ \{\ A_1, ..., A_n\ \} : R} \quad \text{(E-Match)}$$

$$\frac{L; \Theta\Gamma \vdash e \to b : R}{L; \Theta\Gamma \vdash \_ \Rightarrow e \to \_ \to b : \alpha(x_m, T, R)} \quad \text{(E-ArmWildcard)}$$

$$\frac{L; \Theta\Gamma, y \vdash e \to b : R}{L; \Theta\Gamma \vdash y \Rightarrow e \to \_ \to \text{let } y = x_m \text{ in } b : \alpha(x_m, T, R)} \quad \text{(E-ArmVariable)}$$

$$\frac{L; \Theta\Gamma \vdash e \to b : R}{L; \Theta\Gamma \vdash l \Rightarrow e \to l \to b : \alpha(x_m, T, R)} \quad \text{(E-ArmLiteral)}$$

$$\frac{\begin{cases} p = E :: v\ [\ B_1, ..., B_r\ ]\ (y_1, ..., y_m) \\ K_{L(E,v)} = K \\ L; \Theta, B_1, ..., B_r; \Gamma, y_1, ..., y_m \vdash e \to b : R \\ A = K[B_1, ..., B_r](y_1, ..., y_m) \to b \end{cases}}{L; \Theta\Gamma \vdash p \Rightarrow e \to A : \alpha(x_m, T, R)} \quad \text{(E-ArmVariant)}$$

$$\frac{\begin{cases} p = S ::\{\ B_1, ..., B_r, l_1 : y_1, ..., l_m : y_m\ \} \\ K_{L(S)} = K \\ L; \Theta, B_1, ..., B_r; \Gamma, y_1, ..., y_m \vdash e \to b : R \\ A = K[B_1, ..., B_r](y_1, ..., y_m) \to b \end{cases}}{L; \Theta\Gamma \vdash p \Rightarrow e \to A : \alpha(x_m, T, R)} \quad \text{(E-ArmStruct)}$$

## 9.7 Operator Expressions

Operators are aliases for fixed ordinary operation names.

There is no trait, typeclass, or general instance search. An operator is available when the corresponding operation name resolves to a direct named function call whose non-contextual arguments and contextual parameters can be checked.

Primitive operators are not special-cased. For example, `1 + 2` elaborates through the ordinary name `op_add`; the standard prelude defines `op_add` as a generic named function with an `auto` operation parameter.

Recognized operator mappings:

| Operator | Operation name |
|---|---|
| `+` | `op_add` |
| `-` | `op_sub` |
| `*` | `op_mul` |
| `/` | `op_div` |
| `%` | `op_rem` |
| unary `-` | `op_neg` |
| `==` | `op_equal` |
| `!=` | `op_not_equal` |
| `<` | `op_less` |
| `<=` | `op_less_eq` |
| `>` | `op_greater` |
| `>=` | `op_greater_eq` |
| `&&` | `op_and` with a thunked right operand |
| `\|\|` | `op_or` with a thunked right operand |
| `!` | `op_not` |

Table 5: Recognized operator mappings

Operation names such as `op_add` are ordinary value names, not reserved words. User code may define them subject to ordinary binding uniqueness. Lane2 v1 does not allow user-defined operator tokens or user-defined operator mappings.

`&&` and `||` are short-circuit boolean operators. They elaborate through `op_and` and `op_or`, but the right operand is passed as a zero-argument function instead of being evaluated before the operation call.

`a && b` desugars to a call equivalent to `op_and(a, fn() { b }).a || b` follows the same rule with `op_or`. The resulting direct named call may use Contextual Resolution to supply the operation value.

Ordinary calls to `op_and` and `op_or` are strict. Only the `&&` and `||` operator syntaxes thunk the right operand.

Concrete expression precedence, associativity, and unary/binary disambiguation follow the MoonBit parser reference where Lane2 has not deliberately removed a feature.

Strict operators elaborate to ordinary direct named calls. Contextual parameters are already supplied by Contextual Resolution before the call rule applies.

$$\frac{\begin{cases} O_{L(o)} = f \\ L; \Theta\Gamma \vdash f(e_1, e_2) \to b : T \end{cases}}{L; \Theta\Gamma \vdash e_1 o e_2 \to b : T} \quad \text{(E-Operator)}$$

Lazy boolean operators elaborate to operation calls whose right operand is a nullary Buslane function.

$$\frac{\begin{cases} O_{L(o)} = f \\ o \in (\&\&, ||) \\ L; \Theta\Gamma \vdash e_1 \to b_1 : \text{Bool} \\ L; \Theta\Gamma \vdash e_2 \to b_2 : \text{Bool} \end{cases}}{L; \Theta\Gamma \vdash e_1 o e_2 \to f(b_1, \text{fn } () \to \text{Bool } \{\ b_2\ \}) : \text{Bool}} \quad \text{(E-LazyOperator)}$$

Unary operators elaborate to ordinary direct named calls with one argument.

$$\frac{\begin{cases} O_{L(o)} = f \\ L; \Theta\Gamma \vdash f(e) \to b : T \end{cases}}{L; \Theta\Gamma \vdash o e \to b : T} \quad \text{(E-UnaryOperator)}$$

Unsafe builtin expressions elaborate to external Buslane value identities selected by the expected type.

$$\frac{X_{L(s,T)} = f \quad M_{V(f)} = (\kappa_e, T)}{L; \Theta\Gamma \vdash \text{builtin } (\ s\ ) \to f : T} \quad \text{(E-Builtin)}$$

# 10 Buslane Core Language

Buslane is the Lane2 Core Language. It is a typed expression-tree language produced after Checked Source and before administrative normalization. ANF is a later intermediate representation, not the Core Language.

Buslane owns its own type objects, identities, metadata, expressions, literals, diagnostics, and verifier rules. Buslane does not depend on source syntax, source names, source spans, checked-source AST nodes, compiler-front-end symbol tables, or compiler-front-end type objects.

## 10.1 Syntax

This section specifies Buslane as an abstract language. Concrete implementations may choose different record names, map layouts, or arena representations, but they must preserve the abstract structure and judgments specified here.

| | |
|---|---|
| $M$ | Buslane metadata registry |
| $G$ | top-level runtime environment |
| $x, y, f$ | Buslane value identities |
| $C$ | Buslane type constructor identity |
| $A, B$ | Buslane type parameter identities |
| $K$ | Buslane data-constructor identity |
| $T, U, R$ | Buslane types |
| $e, c, a, b$ | Buslane expressions |
| $w$ | Buslane runtime values |
| $r$ | recursive binding group |
| $l$ | primitive literal |
| $\mathcal{D}(G)$ | domain of a runtime environment |
| $M \vdash x : T$ | metadata gives value identity $x$ type $T$ |
| $M \vdash A : K$ | metadata gives type parameter identity $A$ kind $K$ |
| $M; \Theta\Gamma \vdash e : T$ | Buslane expression typing |
| $G \vdash e \to e'$ | one small-step evaluation step |
| $e[T/A]$ | capture-avoiding type substitution |
| $e[w/x]$ | capture-avoiding value substitution |
| $\tau_{C(A_1,\ldots,A_n;K_1,\ldots,K_m)}$ | type-constructor metadata shape |
| $\tau_{K(C;B_1,\ldots,B_r;P_1,\ldots,P_m)}$ | data-constructor metadata shape |
| $\pi_{K(M,K,T_1,\ldots,T_n;U_1,\ldots,U_r)} = (Q_1, \ldots, Q_m)$ | instantiated data-constructor payload sequence |
| $\alpha(x, T, R)$ | alternative type for scrutinee binder $x$, scrutinee type $T$, and result type $R$ |
| $\chi(M, T, a_1, \ldots, a_n)$ | exhaustive alternative sequence |
| $\tau_{l(l)}$ | primitive literal type |
| $\rho_{r(r)}$ | recursive-group substitution |
| $\delta_{l(l,a_1,\ldots,a_n)} = b$ | literal alternative selection |
| $\delta_{K(K,a_1,\ldots,a_n)} = A$ | data-constructor alternative selection |
| $\delta_0(a_1, \ldots, a_n) = b$ | default alternative selection |
| $\nu_0(w, a_1, \ldots, a_n)$ | no literal or data alternative applies to value $w$ |

Table 6: Buslane notation

**Program form.**

$$P ::= (M; d_1, \ldots, d_n)$$

The metadata component $M$ contains all Buslane type, type-parameter, value, and data-constructor metadata. The top-level sequence $d_1, \ldots, d_n$ preserves top-level value initialization order.

Buslane metadata is not required to be dead-code-free. A program may contain well-formed metadata entries that are not referenced by the top-level term sequence.

**Identities.**

$$C \in I_C \quad A, B \in I_A \quad x, y, f \in I_x \quad K \in I_K$$

Buslane has no field identity, source variant identity, source name, or display-name identity. Source structs and enum variants are lowered to nominal data constructors before Buslane. Source field access is lowered to ordinary selector function calls before Buslane.

**Metadata.**

$$\begin{cases} M = (M_V, M_A, M_C, M_K) \\ M_{V(x)} = (\kappa_V, T) \\ \kappa_V ::= \kappa_f \mid \kappa_v \mid \kappa_e \mid \kappa_p \mid \kappa_l \mid \kappa_m \\ M_{A(A)} = K \\ M_{C(C)} = (A_1, ..., A_n; K_1, ..., K_m) \\ M_{K(K)} = (C; B_1, ..., B_r; P_1, ..., P_m) \end{cases}$$

$\kappa_V$ is a Buslane binding category. It is not source syntax, visibility, source origin, or generated/source-written status.

In $M_{K(K)}$, $B_1, ..., B_r$ are the hidden type parameters introduced by data constructor $K$. Universal type parameters belong to the owner nominal type $C$.

**Types.**

$$\begin{cases} T ::= \text{Unit} \mid \text{Bool} \mid \text{Int} \mid \text{String} \\ \quad \mid A \\ \quad \mid C[T_1, ..., T_n] \\ \quad \mid (T_1, ..., T_n) \to R \\ \quad \mid \forall A_1, ..., A_n.T \end{cases}$$

Buslane v1 has only the kind `Type`, but type-parameter metadata records kinds so that the representation remains higher-kind-ready. Buslane has no type-level lambda in v1.

Buslane has no standalone `Exists` type constructor. Existential information is nominal: hidden type members are recorded in data-constructor metadata, construction supplies type witnesses, and matches introduce abstract type binders when opening hidden members.

**Literals.**

$$l ::= () \mid \text{true} \mid \text{false} \mid i \mid s$$

Integer literal values $i$ are normalized `Int64` values. String literal values $s$ are normalized ASCII byte sequences.

**Top-level terms.**

$$\begin{cases} d ::= \text{let } x = e \\ \quad \mid \text{letrec } r \\ \quad \mid \text{external } x \\ r ::= (x_1 = h_1), ..., (x_n = h_n) \end{cases}$$

Types and data constructors live in metadata, not in the top-level term sequence. Top-level pattern declarations do not enter Buslane.

letrec $r$ represents recursive function groups. Buslane well-formedness does not require such groups to be minimal strongly connected components; minimal grouping is a Lane lowering quality property.

**Expressions.**

$$
\begin{cases}
e ::= x \\
\quad \mid l \\
\quad \mid \text{fn } (x_1, ..., x_n) \to R \ \{ \ e \ \} \\
\quad \mid e(e_1, ..., e_n) \\
\quad \mid \text{fn } [ \ A_1, ..., A_n \ ] \ \{ \ e \ \} \\
\quad \mid e[T_1, ..., T_n] \\
\quad \mid K[T_1, ..., T_n; U_1, ..., U_r](e_1, ..., e_m) \\
\quad \mid \text{let } x = e_1 \text{ in } e_2 \\
\quad \mid \text{letrec } r \text{ in } e \\
\quad \mid \text{match } e \text{ as } x \to R \ \{ \ a_1, ..., a_n \ \}
\end{cases}
$$

$$
\begin{cases}
a ::= \_ \to e \\
\quad \mid l \to e \\
\quad \mid K[B_1, ..., B_r](y_1, ..., y_m) \to e
\end{cases}
$$

The variable form $x$ is the only value-reference form. Parameters, locals, top-level values, functions, match binders, and external values are distinguished by metadata and scope rules, not by separate expression variants.

Buslane has no `if` expression form, field-access expression form, unsafe-builtin expression form, cast form, coercion form, source-note form, profiling form, debug annotation form, or ANF atom/RHS category.

## 10.2 Typing

Buslane typing is defined over a complete program. A conforming implementation must provide program-level verification. Expression and type verification may exist as internal operations, but the public semantic boundary is a complete Buslane program.

**Program verification.**

$$
\frac{\begin{cases} P = (M; d_1, ..., d_n) \\ \forall i. M \vdash d_i \checkmark \end{cases}}{\vdash P \checkmark} \quad \text{(B-T-Program)}
$$

Verifier diagnostics report Buslane identities, node paths, and structural errors. They do not contain source spans. User-facing diagnostics recover source locations through an origin side table outside Buslane.

Top-level terms are verified against metadata:

$$
\frac{M \vdash x : T \quad M; (); () \vdash e : T}{M \vdash \text{let } x = e \checkmark} \quad \text{(B-T-TopLet)}
$$

$$
\frac{\begin{cases} r = (x_1 = h_1), ..., (x_n = h_n) \\ \forall i. M \vdash x_i : T_i \\ \forall i. M; (); x_1, ..., x_n \vdash h_i : T_i \end{cases}}{M \vdash \text{letrec } r \checkmark} \quad \text{(B-T-TopLetRec)}
$$

$$
\frac{M_{V(x)} = (\kappa_e, T)}{M \vdash \text{external } x \checkmark} \quad \text{(B-T-TopExternal)}
$$

**Type well-formedness.**

$$
M; \Theta \vdash \text{Unit} : \text{Type}
$$

$$
M; \Theta \vdash \text{Bool} : \text{Type}
$$

$$M; \Theta \vdash \text{Int} : \text{Type}$$

$$M; \Theta \vdash \text{String} : \text{Type}$$

$$\frac{A \in \Theta \quad M \vdash A : \text{Type}}{M; \Theta \vdash A : \text{Type}} \quad \text{(B-T-Parameter)}$$

$$\frac{\begin{cases} M \vdash C : \tau_{C(A_1,...,A_n;K_1,...,K_m)} \\ \forall i. M; \Theta \vdash T_i : \text{Type} \end{cases}}{M; \Theta \vdash C[T_1, ..., T_n] : \text{Type}} \quad \text{(B-T-Nominal)}$$

$$\frac{\begin{cases} \forall i. M; \Theta \vdash T_i : \text{Type} \\ M; \Theta \vdash R : \text{Type} \end{cases}}{M; \Theta \vdash (T_1, ..., T_n) \to R : \text{Type}} \quad \text{(B-T-FunType)}$$

$$\frac{\begin{cases} \forall i. M \vdash A_i : \text{Type} \\ M; \Theta, A_1, ..., A_n \vdash T : \text{Type} \end{cases}}{M; \Theta \vdash \forall A_1, ..., A_n. T : \text{Type}} \quad \text{(B-T-Forall)}$$

Forall type equality uses alpha-equivalence. Globally unique `TypeParameterId` values are a representation and scoping device; raw binder identity equality is not the equality rule for forall types.

**Value reference.**

$$\frac{x \in \Gamma \quad M \vdash x : T}{M; \Theta\Gamma \vdash x : T} \quad \text{(B-T-Ref)}$$

**Literals.**

$$M; \Theta\Gamma \vdash () : \text{Unit}$$

$$M; \Theta\Gamma \vdash b : \text{Bool}$$

$$M; \Theta\Gamma \vdash i : \text{Int}$$

$$M; \Theta\Gamma \vdash s : \text{String}$$

**Function introduction.**

$$\frac{\begin{cases} \forall i. M \vdash x_i : T_i \\ M; \Theta\Gamma, x_1, ..., x_n \vdash e : R \end{cases}}{M; \Theta\Gamma \vdash \text{fn } (x_1, ..., x_n) \to R \ \{ \ e \ \} : (T_1, ..., T_n) \to R} \quad \text{(B-T-Fun)}$$

The function body is an arbitrary Buslane expression checked against the explicit result type. Parameter types come from metadata.

**Call elimination.**

$$\frac{\begin{cases} M; \Theta\Gamma \vdash f : (T_1, ..., T_n) \to R \\ \forall i. M; \Theta\Gamma \vdash a_i : T_i \end{cases}}{M; \Theta\Gamma \vdash f(a_1, ..., a_n) : R} \quad \text{(B-T-Call)}$$

Calls do not store result types. The result type is synthesized from the callee function type.

**Type lambda introduction.**

$$\frac{\begin{cases}\forall i. M \vdash A_i : \text{Type} \\ M; \Theta, A_1, ..., A_n; \Gamma \vdash e : T\end{cases}}{M; \Theta\Gamma \vdash \text{fn } [\ A_1, ..., A_n\ ]\ \{\ e\ \} : \forall A_1, ..., A_n.T} \quad \text{(B-T-TAbs)}$$

**Type application elimination.**

$$\frac{\begin{cases}M; \Theta\Gamma \vdash f : \forall A_1, ..., A_n.T \\ \forall i. M; \Theta \vdash U_i : \text{Type}\end{cases}}{M; \Theta\Gamma \vdash f[U_1, ..., U_n] : T[U_1/A_1, ..., U_n/A_n]} \quad \text{(B-T-TApp)}$$

**Data construction.**

$$\frac{\begin{cases}M \vdash K : \tau_{K(C;B_1,...,B_r;P_1,...,P_m)} \\ \pi_{K(M,K,T_1,...,T_n;U_1,...,U_r)} = (Q_1, ..., Q_m) \\ \forall i. M; \Theta\Gamma \vdash e_i : Q_i\end{cases}}{M; \Theta\Gamma \vdash K\ [\ T_1, ..., T_n; U_1, ..., U_r\ ]\ (e_1, ..., e_m) : C[T_1, ..., T_n]} \quad \text{(B-T-Construct)}$$

**Let.**

$$\frac{\begin{cases}M \vdash x : T \\ M; \Theta\Gamma \vdash e_1 : T \\ M; \Theta\Gamma, x \vdash e_2 : R\end{cases}}{M; \Theta\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : R} \quad \text{(B-T-Let)}$$

`Let` may bind an already-polymorphic value. Buslane does not perform implicit let-generalization.

**Let-rec.**

$$\frac{\begin{cases}\forall i. M \vdash x_i : T_i \\ \forall i. M; \Theta\Gamma, x_1, ..., x_n \vdash h_i : T_i \\ M; \Theta\Gamma, x_1, ..., x_n \vdash e : R\end{cases}}{M; \Theta\Gamma \vdash \text{letrec } (x_1 = h_1, ..., x_n = h_n) \text{ in } e : R} \quad \text{(B-T-LetRec)}$$

Every right-hand side $h_i$ in a let-rec group must be a function value or a type-lambda-wrapped function value. General recursive values are invalid.

**Match.**

$$\frac{\begin{cases}M; \Theta\Gamma \vdash e : T \\ M \vdash x : T \\ \forall i. M; \Theta\Gamma \vdash a_i : \alpha(x, T, R) \\ \chi(M, T, a_1, ..., a_n)\end{cases}}{M; \Theta\Gamma \vdash \text{match } e \text{ as } x \to R\ \{\ a_1, ..., a_n\ \} : R} \quad \text{(B-T-Match)}$$

Alternative bodies do not store result types. Each body is checked against the enclosing match result type.

The default alternative is valid for any scrutinee type:

$$\frac{M; \Theta\Gamma, x \vdash b : R}{M; \Theta\Gamma \vdash \_ \to b : \alpha(x, T, R)} \quad \text{(B-T-AltDefault)}$$

Literal alternatives are valid when the literal type equals the scrutinee type:

$$\frac{\begin{cases}\tau_{l(l)} \equiv T \\ M; \Theta\Gamma, x \vdash b : R\end{cases}}{M; \Theta\Gamma \vdash l \to b : \alpha(x, T, R)} \quad \text{(B-T-AltLit)}$$

Data-constructor alternatives open hidden type binders and bind payload values positionally:

$$\frac{\begin{cases}T \equiv C[T_1, ..., T_n] \\ \pi_{K(M,K,T_1,...,T_n;B_1,...,B_r)} = (Q_1, ..., Q_m) \\ \forall i. M \vdash y_i : Q_i \\ M; \Theta, B_1, ..., B_r; \Gamma, x, y_1, ..., y_m \vdash b : R\end{cases}}{M; \Theta\Gamma \vdash K\ [\ B_1, ..., B_r\ ]\ (y_1, ..., y_m) \to b : \alpha(x, T, R)} \quad \text{(B-T-AltData)}$$

A match must be exhaustive, may contain at most one default alternative, and any default alternative must be last. Duplicate literal alternatives and duplicate data-constructor alternatives are invalid.

## 10.3 Evaluation

Buslane dynamic semantics are specified by small-step reduction over Buslane expressions. The relation $G \vdash e \to e'$ means expression $e$ takes one step to $e'$ under top-level runtime environment $G$.

Runtime values are:

$$\begin{cases}w ::= l \\ \quad \mid \text{fn}\ (x_1, ..., x_n) \to R\ \{\ e\ \} \\ \quad \mid \text{fn}\ [\ A_1, ..., A_n\ ]\ \{\ e\ \} \\ \quad \mid \langle K[T_1, ..., T_n; U_1, ..., U_r](w_1, ..., w_m)\rangle \\ \quad \mid \text{rec}(r, x) \\ \quad \mid \text{external}\ x\end{cases}$$

Type arguments are runtime-erased for ordinary computation, but they remain present in the dynamic notation so that type-lambda elimination and existential opening are specified precisely.

**Evaluation contexts.** Buslane evaluates strictly from left to right.

$$\begin{cases}E ::= [] \\ \quad \mid E(e_1, ..., e_n) \\ \quad \mid w_0(..., w_{i-1}, E, e_{i+1}, ..., e_n) \\ \quad \mid E[T_1, ..., T_n] \\ \quad \mid K[T; U](w_1, ..., w_{i-1}, E, e_{i+1}, ..., e_m) \\ \quad \mid \text{let}\ x = E\ \text{in}\ e \\ \quad \mid \text{match}\ E\ \text{as}\ x \to R\ \{\ a_1, ..., a_n\ \}\end{cases}$$

$$\frac{G \vdash e \to e'}{G \vdash E[e] \to E[e']} \quad \text{(B-E-Ctx)}$$

**Top-level reference.**

$$\frac{G(x) = w}{G \vdash x \to w} \quad \text{(B-E-TopRef)}$$

Local binders are eliminated by capture-avoiding substitution rules below.

**Let.**

$$\frac{}{G \vdash \text{let}\ x = w\ \text{in}\ e \to e[w/x]} \quad \text{(B-E-Let)}$$

**Let-rec.**

$$\frac{\begin{cases} r = ((x_1 = h_1), ..., (x_n = h_n)) \\ \rho_{r(r)} = [\mathrm{rec}(r, x_1)/x_1, ..., \mathrm{rec}(r, x_n)/x_n] \end{cases}}{G \vdash \text{letrec } r \text{ in } e \to e\rho_{r(r)}} \quad \text{(B-E-LetRec)}$$

Recursive values unroll when forced:

$$\frac{\begin{cases} r = ((x_1 = h_1), ..., (x_n = h_n)) \\ \rho_{r(r)} = [\mathrm{rec}(r, x_1)/x_1, ..., \mathrm{rec}(r, x_n)/x_n] \end{cases}}{G \vdash \mathrm{rec}(r, x_i) \to h_i\rho_{r(r)}} \quad \text{(B-E-Rec)}$$

**Function call.**

$$\frac{\sigma = [w_1/x_1, ..., w_n/x_n]}{G \vdash (\text{fn } (x_1, ..., x_n) \to R \ \{ \ e \ \})(w_1, ..., w_n) \to e\sigma} \quad \text{(B-E-Call)}$$

Function arguments are evaluated before this rule applies.

**Type application.**

$$\frac{\sigma = [T_1/A_1, ..., T_n/A_n]}{G \vdash (\text{fn } [ \ A_1, ..., A_n \ ] \ \{ \ e \ \}) \ [ \ T_1, ..., T_n \ ] \to e\sigma} \quad \text{(B-E-TApp)}$$

**Data construction.**

$$\frac{\begin{cases} c = K \ [ \ T; U \ ] \ (w) \\ d = \langle K \ [ \ T; U \ ] \ (w) \rangle \end{cases}}{G \vdash c \to d} \quad \text{(B-E-Construct)}$$

Here, `T`, `U`, and `w` abbreviate the constructor's owner type arguments, hidden witness types, and evaluated payload values.

**Literal match.**

$$\frac{\delta_{l(l, a_1, ..., a_n)} = b}{G \vdash \text{match } l \text{ as } x \to R \ \{ \ a_1, ..., a_n \ \} \to b[l/x]} \quad \text{(B-E-MatchLit)}$$

**Data-constructor match.**

$$\frac{\begin{cases} d = \langle K \ [ \ T; U_1, ..., U_r \ ] \ (w_1, ..., w_m) \rangle \\ A = K \ [ \ B_1, ..., B_r \ ] \ (y_1, ..., y_m) \to b \\ \delta_{K(K, a_1, ..., a_n)} = A \\ \rho = [U_1/B_1, ..., U_r/B_r] \\ \sigma = [d/x, w_1/y_1, ..., w_m/y_m] \end{cases}}{G \vdash \text{match } d \text{ as } x \to R \ \{ \ a_1, ..., a_n \ \} \to b\rho\sigma} \quad \text{(B-E-MatchData)}$$

**Default match.**

$$\frac{\begin{cases} \delta_0(a_1, ..., a_n) = b \\ \nu_0(w, a_1, ..., a_n) \end{cases}}{G \vdash \text{match } w \text{ as } x \to R \ \{ \ a_1, ..., a_n \ \} \to b[w/x]} \quad \text{(B-E-MatchDefault)}$$

Because Buslane matches are verified as exhaustive, a well-formed Buslane match over a safe value never gets stuck for lack of an alternative.

**External values.** External values are supplied by the execution environment. Applying an external function value delegates to the runtime contract associated with that value. Misuse of an external builtin is undefined behavior.

### 10.4 Relation to ANF

ANF preserves Buslane typing and evaluation behavior while introducing atom, right-hand-side, binding, and temporary structure to make evaluation order explicit. A conforming implementation may evaluate Buslane directly or lower Buslane to ANF first. The observable behavior of safe programs must be the same.

## 11 Evaluation

Lane2 v1 is pure and strict.

- Evaluating a safe expression depends only on its inputs and produces a value.
- Function arguments are evaluated before the function body runs.
- `if` and `match` evaluate only the selected branch.
- There is no mutable binding, mutable field, assignment, field update, or implicit statement sequencing.
- IO and other effects are not part of v1 core semantics.

The `Unit` value is written explicitly as `()`.

Empty blocks are invalid. A block must contain exactly one final expression after any local items.

Function calls evaluate all arguments before entering the function body. The only v1 operator forms that delay a subexpression are `&&` and `||`, which pass the right operand as a thunk to the resolved `op_and` or `op_or` value.

## 12 Undefined Behavior

Incorrect builtin use can produce undefined behavior. Lane2′s type safety guarantee applies only to programs that do not misuse `builtin`.

Invalid integer arithmetic is undefined behavior in v1. This includes signed 64-bit overflow, division by zero, remainder by zero, `MIN_INT / -1`, and negating `MIN_INT`.

## 13 Conformance

The words “must”, “must not”, “may”, and “should” are normative in this document.

Text marked as rationale, examples, or notes is informative unless it explicitly uses normative language.

An implementation conforms to this draft if it accepts all valid programs described here, rejects invalid programs described here, and gives the specified typing and evaluation behavior for safe programs.

Programs that misuse `builtin` are outside Lane2′s safety guarantee.

## 14 Appendix

### 14.1 Deliberately Omitted From V1

V1 omits:

- mutation and assignment,
- field update and copy update,
- modules and imports,
- local type definitions,
- labeled function parameters,
- labeled enum payloads,
- type aliases,
- traits, typeclasses, interfaces,
- method syntax,
- tuple types,
- collections,
- pattern guards,

- or-patterns,
- as-patterns,
- `is` pattern expressions,
- placeholder pipeline.